



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 1
Implement Gradient Descent Algorithm
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

2024-2025

Aim: To implement the Gradient Descent algorithm to minimize a given function.

Theory:

Gradient Descent is an optimization algorithm used to minimize the loss function in machine learning models. It works by iteratively adjusting the parameters of the model in the direction of the negative gradient of the loss function with respect to those parameters.

- Types: Batch Gradient Descent (uses the entire dataset), Stochastic Gradient Descent (uses one sample at a time), and Mini-batch Gradient Descent (uses small batches).

- Key Formula:

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$$

Here, η is the learning rate, and $\nabla f(\theta_t)$ is the gradient of the loss function at the current parameter value.

- Challenges: Proper tuning of the learning rate and handling non-convex functions with local minima or saddle points.

Procedure/Algorithm:

1. Define the function to be minimized.
2. Initialize parameters (weights) randomly.
3. Set the learning rate and the number of iterations.
4. Repeat until convergence:
 - Compute the gradient of the function with respect to the parameters.
 - Update parameters using the gradient and learning rate.
5. Display the optimized parameters and the value of the function.

Code:

$f(x) = x^2 + 3x + 5$ is the function we want to minimize.

$\text{gradient}(x) = 2x + 3$ is the derivative of $f(x)$. It tells us which direction to move to decrease the function's value.

```
def f(x):
```

```
    return x**2 + 3*x + 5
```

```
def gradient(x):
```

```
    return 2*x + 3
```

```
def gradient_descent(x, learning_rate, iterations):
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
for i in range(1, iterations + 1):
```

```
    grad = gradient(x)
    x = x - learning_rate * grad
```

```
    if i <= 10 or i == iterations:
        print(f"Iteration {i}: x = {x}, f(x) = {f(x)}")
```

```
return x
```

```
x_start = float(input("Enter starting x value: "))
learning_rate = float(input("Enter learning rate (e.g., 0.1): "))
iterations = int(input("Enter number of iterations: "))
```

```
optimized_x = gradient_descent(x_start, learning_rate, iterations)
print(f"\nFinal optimized x = {optimized_x}")
print(f"Function value at optimized x: f({optimized_x}) = f(optimized_x)
```

Output:

```
Enter starting x value: 2
Enter learning rate (e.g., 0.1): 0.1
Enter number of iterations: 10
Iteration 1: x = 1.2999999999999998, f(x) = 10.59
Iteration 2: x = 0.7399999999999999, f(x) = 7.7676
Iteration 3: x = 0.2919999999999999, f(x) = 5.961264
Iteration 4: x = -0.06640000000000007, f(x) = 4.80520896
Iteration 5: x = -0.35312000000000001, f(x) = 4.065333734399999
Iteration 6: x = -0.58249600000000001, f(x) = 3.5918135900159998
Iteration 7: x = -0.76599680000000001, f(x) = 3.2887606976102397
Iteration 8: x = -0.91279744000000001, f(x) = 3.094806846470554
Iteration 9: x = -1.030237952, f(x) = 2.9706763817411543
Iteration 10: x = -1.1241903616, f(x) = 2.891232884314339
```

```
Final optimized x = -1.1241903616
Function value at optimized x: f(-1.1241903616) = 2.891232884314339
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Conclusion: Answer the following questions

1. What is the role of the learning rate?

The **learning rate** controls how big or small the steps are when updating the value of x in Gradient Descent. It decides how quickly the algorithm moves toward the minimum of the function. A good learning rate helps the algorithm reach the lowest point efficiently without taking too many steps or overshooting the minimum.

2. What happens if the learning rate is too high or too low?

If the learning rate is too high, the steps will be too big, and the algorithm might skip over the minimum or never settle down, causing it to diverge (never find the best value). If the learning rate is too low, the steps will be too small, and the algorithm will take a very long time to reach the minimum, making it slow and inefficient. A balanced learning rate is needed for fast and accurate optimization.